

# EXPERIMENT 1

## Bank Account Management System using Classes and Objects

### ■ Basic Information

**Experiment No:** 01

**Date:** 11/09/2026

**Title:** Bank Account Management System using Classes and Objects

### ■ Objective / Aim

To develop a **Bank Account Management System** in Java using **classes and objects**, demonstrating inheritance, method overriding, deposit, withdrawal, interest calculation, fund transfer, and account information display.

### ■ System Requirements

#### Hardware

- Standard PC/Laptop
- Minimum 4 GB RAM
- Minimum 500 MB free storage

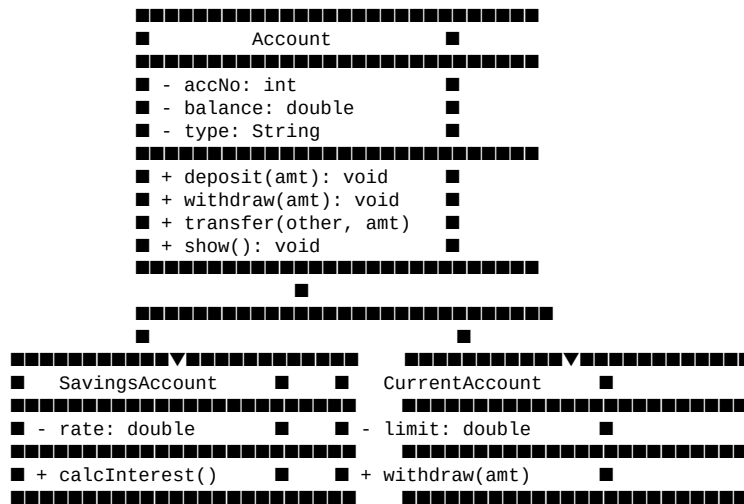
#### Software

- Windows / Linux / macOS
- Java Development Kit (JDK)
- Java IDE such as Eclipse, IntelliJ IDEA, NetBeans, or VS Code
- Command Prompt / Terminal

### ■ Algorithm / Logic Description

1. Start the program.
2. Import the Scanner class for taking input from the user.
3. Create a base class named Account.
4. Define account number, balance, and account type as data members.
5. Define a constructor to initialize the account details.
6. Create a deposit() method to add money to the account.
7. Create a withdraw() method to withdraw money if sufficient balance is available.
8. Create a transfer() method to transfer money from one account to another.
9. Create a show() method to display account details.
10. Create a SavingsAccount class that inherits from Account.
11. Add an interest rate to the savings account and calculate interest using calcInterest().
12. Create a CurrentAccount class that inherits from Account.
13. Add an overdraft limit to the current account.
14. Override the withdraw() method in CurrentAccount to allow withdrawal up to the overdraft limit.
15. Create objects for savings and current accounts.
16. Display a menu containing deposit, withdrawal, interest, transfer, account details, and exit options.
17. Read the user's choice using Scanner.
18. Execute the selected operation using a switch statement.
19. Continue displaying the menu until the user selects Exit.
20. Close the scanner and stop the program.

## ■ UML Class Diagram



**Relationship:** SavingsAccount and CurrentAccount inherit from the Account class.

## ■ Source Code

**File name:** Experiment1.java

```
package recordprog;

import java.util.Scanner;

//Base class for all accounts
class Account {

    int accNo;        // account number
    double balance;   // money in account
    String type;      // savings or current

    Account(int no, double bal, String t) {
        accNo = no;
        balance = bal;
        type = t;
    }

    // deposit money
    void deposit(double amt) {
        balance += amt;
        System.out.println("Deposited: " + amt);
        System.out.println("Balance: " + balance);
    }

    // withdraw money
    void withdraw(double amt) {
        if (amt <= balance) {
            balance -= amt;
            System.out.println("Withdrawn: " + amt);
            System.out.println("Balance: " + balance);
        } else {
            System.out.println("Not enough balance!");
        }
    }

    // transfer money to another account
    void transfer(Account other, double amt) {
        if (amt <= balance) {
            balance -= amt;
            other.balance += amt;
            System.out.println("Transfer done!");
        } else {
            System.out.println("Transfer failed!");
        }
    }

    // show account info
    void show() {
        System.out.println("Acc No: " + accNo);
        System.out.println("Type : " + type);
        System.out.println("Bal : " + balance);
    }
}

//Savings account class
class SavingsAccount extends Account {

    double rate; // interest rate

    SavingsAccount(int no, double bal, double r) {
        super(no, bal, "Savings");
        rate = r;
    }

    void calcInterest() {
        double interest = balance * rate / 100;
        System.out.println("Interest: " + interest);
    }
}

//Current account class
class CurrentAccount extends Account {

    double limit; // overdraft limit

    CurrentAccount(int no, double bal, double l) {
        super(no, bal, "Current");
        limit = l;
    }

    // overriding withdraw
    @Override
```

```

    void withdraw(double amt) {
        if (amt <= balance + limit) {
            balance -= amt;
            System.out.println("Withdraw ok. Balance: " + balance);
        } else {
            System.out.println("Overdraft limit crossed!");
        }
    }
}

//main class
public class Experiment1 {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        SavingsAccount sAcc =
            new SavingsAccount(101, 5000, 5);

        CurrentAccount cAcc =
            new CurrentAccount(201, 3000, 2000);

        int ch;

        do {

            System.out.println("\n--- Bank Menu ---");
            System.out.println("1. Deposit (Savings)");
            System.out.println("2. Withdraw (Savings)");
            System.out.println("3. Interest (Savings)");
            System.out.println("4. Withdraw (Current)");
            System.out.println("5. Transfer Savings -> Current");
            System.out.println("6. Show Accounts");
            System.out.println("7. Exit");

            System.out.print("Choice: ");
            ch = sc.nextInt();

            switch (ch) {

                case 1:
                    System.out.print("Amount: ");
                    sAcc.deposit(sc.nextDouble());
                    break;

                case 2:
                    System.out.print("Amount: ");
                    sAcc.withdraw(sc.nextDouble());
                    break;

                case 3:
                    sAcc.calcInterest();
                    break;

                case 4:
                    System.out.print("Amount: ");
                    cAcc.withdraw(sc.nextDouble());
                    break;

                case 5:
                    System.out.print("Amount: ");
                    sAcc.transfer(cAcc, sc.nextDouble());
                    break;

                case 6:
                    System.out.println("Savings:");
                    sAcc.show();

                    System.out.println("Current:");
                    cAcc.show();
                    break;

                case 7:
                    System.out.println("Bye!");
                    break;

                default:
                    System.out.println("Wrong choice!");
            }

        } while (ch != 7);

        sc.close();
    }
}

```

## ■ Compilation & Execution Commands

Since the program contains package `recordprog`;, save the file inside a folder named `recordprog`.  
`recordprog/Experiment1.java`

### Compile:

```
javac recordprog/Experiment1.java
```

### Run:

```
java recordprog.Experiment1
```

## ■■ Sample Output

```
--- Bank Menu ---
1. Deposit (Savings)
2. Withdraw (Savings)
3. Interest (Savings)
4. Withdraw (Current)
5. Transfer Savings -> Current
6. Show Accounts
7. Exit
Choice: 1
Amount: 2000
Deposited: 2000.0
Balance: 7000.0
```

```
--- Bank Menu ---
1. Deposit (Savings)
2. Withdraw (Savings)
3. Interest (Savings)
4. Withdraw (Current)
5. Transfer Savings -> Current
6. Show Accounts
7. Exit
Choice: 2
Amount: 1000
Withdrawn: 1000.0
Balance: 6000.0
```

```
--- Bank Menu ---
1. Deposit (Savings)
2. Withdraw (Savings)
3. Interest (Savings)
4. Withdraw (Current)
5. Transfer Savings -> Current
6. Show Accounts
7. Exit
Choice: 3
Interest: 300.0
```

```
--- Bank Menu ---
1. Deposit (Savings)
2. Withdraw (Savings)
3. Interest (Savings)
4. Withdraw (Current)
5. Transfer Savings -> Current
6. Show Accounts
7. Exit
Choice: 6
Savings:
Acc No: 101
Type : Savings
Bal : 6000.0
Current:
Acc No: 201
Type : Current
Bal : 3000.0
```

```
--- Bank Menu ---
1. Deposit (Savings)
2. Withdraw (Savings)
3. Interest (Savings)
4. Withdraw (Current)
5. Transfer Savings -> Current
6. Show Accounts
7. Exit
Choice: 7
Bye!
```

## ■ Result

The **Bank Account Management System** was successfully implemented in Java using **classes, objects, inheritance, and method overriding**. The program performs deposit, withdrawal, interest calculation, fund transfer, and account information display operations successfully.

## Output Screen

```
7. Exit
Choice: 1
Amount: 500
Deposited: 500.0
Balance: 5500.0

--- Bank Menu ---
1. Deposit (Savings)
2. Withdraw (Savings)
3. Interest (Savings)
4. Withdraw (Current)
5. Transfer Savings -> Current
6. Show Accounts
7. Exit
Choice: 2
Amount: 200
Withdrawn: 200.0
Balance: 5300.0

--- Bank Menu ---
1. Deposit (Savings)
2. Withdraw (Savings)
3. Interest (Savings)
4. Withdraw (Current)
5. Transfer Savings -> Current
6. Show Accounts
7. Exit
Choice: 6
Savings:
Acc No: 101
Type : Savings
Bal : 5300.0
Current:
Acc No: 201
Type : Current
Bal : 3000.0
```